

Decentralized, Browser-Based Docker IDEs for C++ and x86-64 Assembly Education: An Evaluation on the Laptops Students Actually Own

Sean Cuenco · Faculty Mentor: James Andro-Vasko, Ph.D. · Department of Computer Science, University of Nevada, Las Vegas
Summer Undergraduate Research Fellowship (SURF) 2026 · Office of Undergraduate Research, UNLV

Code, handouts, test script, raw results:
github.com/seancnc003/UNLV-DockerIDEs

Ready-made course environments:
hub.docker.com/r/seancnc/unlv-cpp-ide
hub.docker.com/r/seancnc/unlv-x86-ide

THE HOOK

Week one of a programming course is often lost to installing software, not to programming. Every year, hundreds of UNLV students in the introductory C++ and assembly-language courses spend their first days making four tools work on dozens of different laptops, and owners of the newest Macs sometimes cannot run the course's software at all. This project asks a simple question: can the whole course environment be one download that works the same on every laptop, and how would we know?

Abstract

UNLV's introductory C++ and x86-64 assembly courses lose their first week to software setup, and the newest Macs often cannot run the course's virtual machine. This project packages each course's exact tools and a browser-based editor into a Docker container that runs entirely on the student's laptop: one 552 MB download, one command, no server, no account. To test whether that promise holds on the laptops students actually own, one unmodified script performed a student's first day and four real assignments on four machines: Intel and ARM Linux rented from AWS, Windows 11 rented from Azure, and an M1 MacBook Pro. All four passed every assignment; the editor opened in 0.5 s on Intel chips and about 5 s on Apple Silicon. The one limit, the debugger on ARM chips, is documented and explained.

Introduction and background

The problem is old and well documented. Other universities measured 75–100 minutes of class time per semester lost to installs [6], help traffic dominated by “it works on my machine” [3], and students with M1/M2 MacBooks unable to run the course's virtual machine [4]. Institutions have answered four ways, and UNLV has lived through the first two:

1 Shared login servers (UNLV, 2000s–2024)

Every student logged in to a department computer over the network (PuTTY). Retired, and the documentation was deleted in 2025 [12].

2 A virtual machine per course (CS 218 today)

A 19 GB copy of a whole computer that runs inside the laptop. Slow, and so slow on Apple Silicon Macs that the course sends M1/M2 owners to the lab.

3 Cloud editors (Harvard CS50 [1])

The environment lives on a company's servers. Needs internet and an account; students never learn to set up their own machine.

4 Monitored platforms (CodeDive, South Korea [8])

The university's cluster runs the editor and records every command and every one-second pause in typing.

Problem and research focus

Before the fellowship I built a fifth option: a container holding each course's exact tools plus the VS Code editor, running entirely on the student's own laptop and opened in a browser tab. One download, one command, no server, no account, nothing recorded. It worked on the laptop I built it on. The research question is whether that promise survives contact with the laptops students actually own:

Q1 Does the same package behave identically on Windows, macOS, and Linux, on Intel and ARM chips, including real CS 218 assignments?

Q2 What does it cost the student: download size, start-up time, memory?

Q3 Where does it break, and can we say exactly why?

Methodology

What we built

Two containers, one per course, each holding the editor and the exact tool versions the instructor chose: the C++ compiler for CS 135; the assembler, linker, and debugger for CS 218. A student installs Docker once, runs the one command below, and opens a browser tab. Coursework stays in a normal folder on the laptop (Figure 1).

```
docker run -p 127.0.0.1:8218:8080 -v ~/UNLV/x86-workspace:/home/coder/workspace seancnc/unlv-x86-ide
```

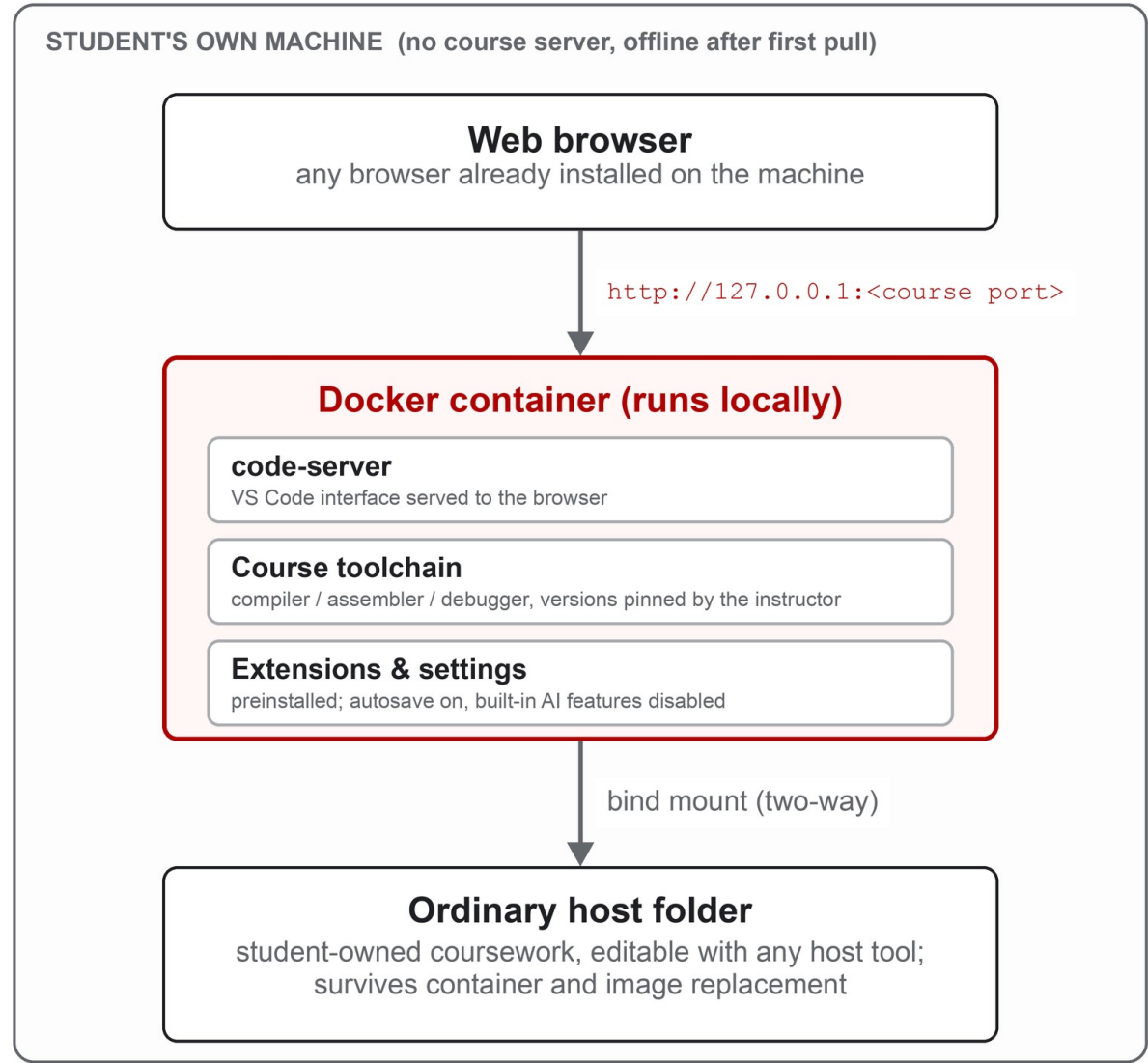


Figure 1. Everything runs on the student's machine. The only time the internet is needed is the first download.

How we tested it: same exam, four kinds of computer

We cannot borrow one of every laptop students own, so we rented them. Three blank machines came from Amazon's and Microsoft's clouds for under \$10 in total; the fourth was a base-model MacBook Pro, the Mac a budget-conscious student buys.

Windows 11 PC	Linux PC	Apple Silicon Mac	Linux on ARM
Intel chip rented (Azure)	Intel chip rented (AWS)	M1 Pro, 16 GB owned	ARM chip rented (AWS)

One script, run unchanged on all four

- Download the package and confirm it is the identical file on every machine.
- Start it and time how long until the editor appears.
- Write, assemble, and run a program three times; try the debugger.
- Build and run four real CS 218 assignments, including a multithreaded one, and measure memory.
- Delete the container and check that the student's files survive.

The script writes every number to a file. Nothing on this poster was typed by hand.

THREE TERMS USED ON THIS POSTER

Container a sealed, pre-configured copy of a course's tools that runs inside any laptop. Docker is the free program that runs containers.

Translation how an ARM chip (Apple Silicon Macs) runs software built for Intel chips. Correct, but slower.

Debugger a tool that pauses a running program so students can look inside it. Some assignments require it.

Results

4 of 4

machines ran every real CS 218 assignment correctly, including the multithreaded one

0.5 s

until the editor opens on Intel chips; about 5 s on Apple Silicon. A course VM takes minutes.

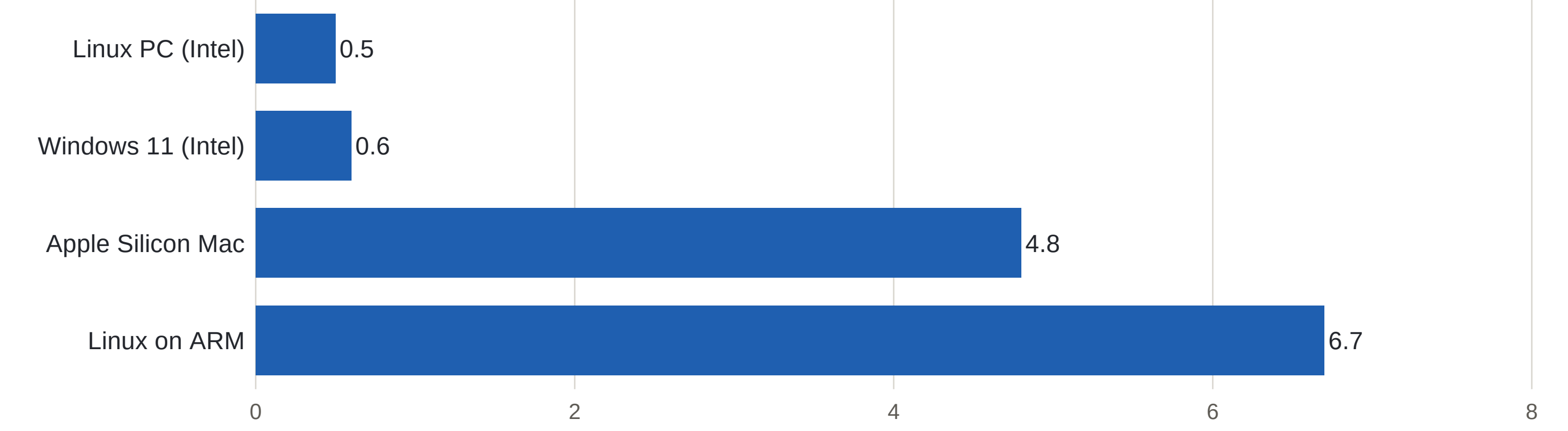
552 MB

one-time download, the identical file on every machine, versus a 19 GB virtual machine

Table 1. What the script recorded on each machine (memory in MiB).

Measure	Windows 11 (Intel)	Linux PC (Intel)	Apple Silicon Mac	Linux on ARM
All four assignments correct	yes	yes	yes	yes
Files survive deleting the container	yes	yes	yes	yes
Debugger	works	works	silent	silent
Editor ready in (seconds)	0.6	0.5	4.8	6.7
Memory while idle	56	54	263	255
Heaviest assignment builds in (s)	0.6	0.3	3.1	3.6
Checks passed / failed	13 / 0	13 / 0	12 / 0	12 / 0

Figure 2. Seconds until the editor is ready



The one limit, in plain words. The assembly course teaches the Intel-style x86-64 instruction set, so that package is built for Intel chips on purpose. Apple Silicon and other ARM chips run it through a translator: every assignment still comes out correct, about ten times slower but still within seconds. The translator cannot support the debugger, so debugger assignments on a Mac must be finished on an Intel machine, and the student handout says so. We also found that the translator that ships with Ubuntu Linux crashes while Docker's own does not; that fix is one line in the handout. Windows and Intel machines, the majority of student laptops, get everything.

Concluding discussion: why this matters

For students

Week one goes to learning, not installing. The same environment on every laptop, offline after one download, with coursework in a normal folder. Nothing is recorded or sent anywhere, and no student is left out because of the laptop they could afford.

For the course and the department

No servers to maintain, no VPN, no 19 GB image to rebuild each semester. The instructor updates one recipe and every student gets the same tools. A trade-off nobody had measured is now explicit: keep the real x86-64 instruction set, and lose the debugger on ARM until a hybrid container is built.

For other courses and researchers

The rent-four-computers, one-script method works for any course environment, and every raw result is public so anyone can check the numbers. The design sits between cloud editors and monitored platforms: the same convenience without the account, the internet dependence, or the surveillance.

What comes next

A classroom study in CS 135 and CS 218 with IRB approval: how many students reach a working setup without help, how much staff time it saves, and how students feel about it [3, 4, 6]. A three-student high-school pilot has already run. Then a hybrid assembly container for ARM that could bring the debugger back to Apple Silicon.